

Do text-to-image models build a 3D model of a world inside? Simple use case with shadows generation

Zaawansowane zagadnienia sieci neuronowych

Dokumentacja wstępna

Natalia Choszczyk | Filip Langiewicz

nr indeksu 327267 | nr indeksu 327293

Politechnika Warszawska

Wydział Elektroniki i Technik Informacyjnych

23 kwietnia 2026 r.

Spis treści

1. Wstęp i cel projektu	3
2. Analiza problemu	4
2.1. Hipoteza badawcza	4
2.2. Scenariusze i oczekiwane wyniki	4
2.3. Kluczowe wyzwania	4
3. Zbiór danych	6
3.1. Generowanie danych	6
3.2. Parametry zbioru danych	7
3.3. Struktura danych JSON	7
3.4. Podział zbioru danych	7
4. Przegląd literatury	8
4.1. Reprezentacje wewnętrzne modeli dyfuzyjnych	8
4.2. Świadomość 3D w modelach wizyjnych	8
4.3. Liniowe sondy jako narzędzie analizy	8
4.4. Architektura modelu SANA	9
5. Plan realizacji projektu	10
5.1. Etap 1 – Przygotowanie zbioru danych	10
5.2. Etap 2 – Konfiguracja środowiska obliczeniowego	10
5.3. Etap 3 – Ekstrakcja aktywacji modelu	10
5.4. Etap 4 – Trening liniowej sondy	11
5.5. Etap 5 – Analiza wyników	11
5.6. Etap 6 – Raport końcowy	11
6. Plan testów i walidacji	12
6.1. Metryki oceny	12
6.2. Punkty odniesienia	12
6.3. Testy	13
7. Wykorzystane technologie	14

1. Wstęp i cel projektu

Nowoczesne modele generatywne typu tekst-obraz potrafią tworzyć realistyczne obrazy zawierające spójne oświetlenie, cienie i głębię przestrzenną. Otwartym pytaniem pozostaje jednak, czy modele te dysponują wewnętrzną reprezentacją trójwymiarowej przestrzeni wraz z rządzącymi nią prawami fizyki, czy też jedynie odwzorowują wyuczone wzorce teksturowe w przestrzeni dwuwymiarowej.

Pośród dostępnych modeli tej klasy, takich jak Stable Diffusion czy FLUX, w projekcie zostanie wykorzystany model SANA-1.6B [2]. Jest to model oparty na architekturze liniowego Diffusion Transformer (DiT), opracowany przez NVIDIA i MIT. Kluczowym uzasadnieniem tego wyboru jest wyjątkowo korzystny stosunek szybkości inferencji do jakości generowanych obrazów: SANA-1.6B osiąga wyniki porównywalne z modelem FLUX-dev na benchmarku DPG-Bench, przy znacznie wyższej przepustowości. Umożliwia to przeprowadzenie ekstrakcji aktywacji dla dużej liczby obrazów w rozsądnym czasie obliczeniowym. Ponadto model jest publicznie dostępny za pośrednictwem biblioteki `diffusers` i może być uruchomiony na standardowych kartach GPU, co upraszcza konfigurację środowiska.

Celem projektu jest empiryczna weryfikacja hipotezy o wewnętrznej reprezentacji 3D z wykorzystaniem analizy mechanistycznej (*mechanistic interpretability*) aktywacji modelu SANA. Zakłada się, że prosta liniowa sonda wytrenowana na zamrożonych aktywacjach pośrednich warstw modelu będzie w stanie przewidywać kierunek źródła światła oraz uogólniać tę predykcję na obiekty nieobecne w zbiorze treningowym.

2. Analiza problemu

2.1. Hipoteza badawcza

Przyjęta hipoteza zakłada, że modele dyfuzyjne w trakcie procesu odszumiania (*denoising*) zapisują w swoich aktywacjach informacje o pewnych cechach fizycznych, takich jak kierunek oświetlenia, a nie tylko dopasowują wzorce tekstur. Oznacza to, że informacje o kącie padania światła mogą być obecne w aktywacjach warstw pośrednich modelu.

2.2. Scenariusze i oczekiwane wyniki

W tabeli 2.1 przedstawiono dwa możliwe scenariusze oraz ich oczekiwany wpływ na jakość predykcji liniowej sondy.

Tabela 2.1: Dwa scenariusze i oczekiwane wyniki sondy liniowej

Scenariusz	Interpretacja	Oczekiwany wynik sondy
Model zawiera reprezentację zależności fizycznych w przestrzeni 3D	Aktywacje zawierają reprezentację kąta oświetlenia	Niska wartość MAE, dobra generalizacja
Model zapamiętuje jedynie wzorce teksturalne w przestrzeni 2D	Aktywacje kodują wzorce pikseli, nie fizyczne zależności	Wysoki błąd, brak generalizacji na nowe obiekty

2.3. Kluczowe wyzwania

Realizacja projektu wiąże się z następującymi wyzwaniami technicznymi:

2.3. KLUCZOWE WYZWANIA

- **Polisemantyczność neuronów:** większość neuronów koduje jednocześnie wiele cech (kształt, kolor, materiał, oświetlenie), co utrudnia izolację informacji dotyczącej wyłącznie kierunku światła.
- **Cykliczność azymutu:** kąt 0° i 360° są identyczne, co wymaga zastosowania specjalnej reprezentacji (pary $(\sin \phi, \cos \phi)$) przy uczeniu regresji.
- **Wybór warstwy:** różne warstwy architektury modelu mogą kodować informacje na odmiennych poziomach abstrakcji; optymalną warstwę należy wybrać eksperymentalnie.
- **Krok czasowy odsumiania:** aktywacje zmieniają się w trakcie procesu *denoising*, dlatego konieczne jest wybranie reprezentatywnych kroków czasowych.

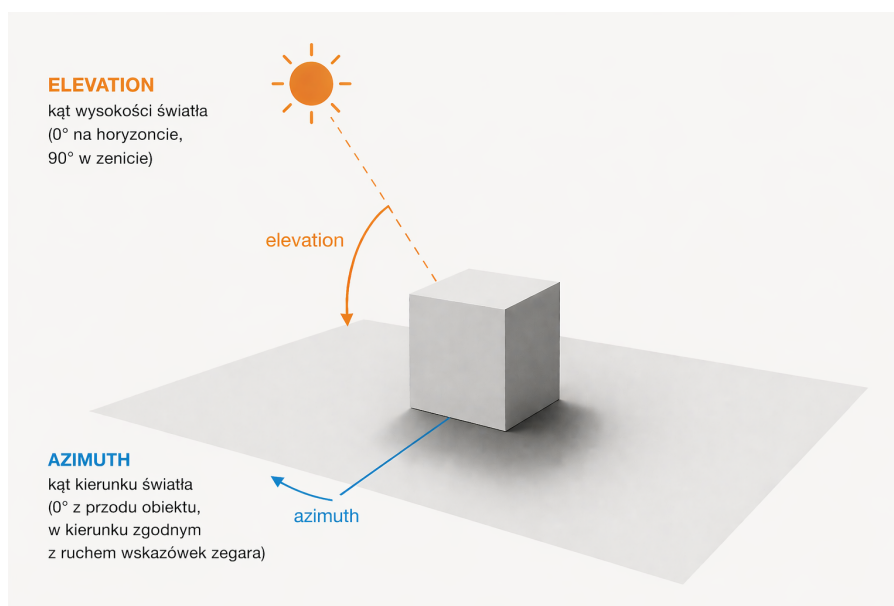
3. Zbiór danych

3.1. Generowanie danych

Zbiór danych został wygenerowany za pomocą zmodyfikowanego skryptu `render_images.py` z repozytorium CLEVR [1], uruchomionego w środowisku Blender 2.78c z silnikiem renderowania Cycles (path tracing). Wprowadzono następujące modyfikacje względem oryginalnego zestawu narzędzi CLEVR:

- **Kontrola oświetlenia:** dodano argumenty `-light_azimuth` oraz `-light_elevation` umożliwiające precyzyjne pozycjonowanie lampy głównej (`Lamp_Key`) w układzie sferycznym.
- **Wyłączenie świateł pomocniczych:** lampy `Lamp_Fill` i `Lamp_Back` zostały ukryte (`hide_render = True`), aby usunąć wpływ innych lamp na generowanie cienia.
- **Jeden obiekt na scenę:** eliminuje interakcje cieni między wieloma obiektami.

Na Rysunku 3.1 przedstawiono definicję kątów azymutu i wysokości wykorzystywanych w układzie sferycznym sceny.



Rysunek 3.1: Definicja kątów azymutu i wysokości w układzie sferycznym

3.2. Parametry zbioru danych

Tabela 3.1: Parametry generowanego zbioru danych

Parametr	Wartość
Rozdzielczość	512×512 px
Próbkowanie	128 próbek/obraz
Azymut	losowy z $\mathcal{U}(0^\circ, 360^\circ)$
Wysokość	losowa z $\mathcal{U}(30^\circ, 75^\circ)$
Liczba obrazów	800
Format pliku	PNG + JSON (metadane)

3.3. Struktura danych JSON

Każdemu obrazowi towarzyszy plik JSON zawierający m.in.:

```
{
  "light_azimuth_deg": 126.2,
  "light_elevation_deg": 59.9,
  "objects": [{ "shape": "cylinder", "material": "rubber",
    "color": "green", ... } ]
}
```

3.4. Podział zbioru danych

Tabela 3.2: Podział zbioru danych według kształtu obiektów

Podzbiór	Kształty	Udział	Cel
Treningowy	walce, kule	$\approx 80\%$	uczenie sondy
Walidacyjny	walce, kule	$\approx 10\%$	dobór hiperparametrów
Testowy	sześciany	$\approx 10\%$	test generalizacji

Podział oparty na kształcie (a nie losowy) pozwala ocenić zdolność sondy do generalizacji na obiekty nieobecne podczas treningu, co stanowi kluczowy test hipotezy badawczej.

4. Przegląd literatury

4.1. Reprezentacje wewnętrzne modeli dyfuzyjnych

Chen et al. [3] wykazali, że aktywacje modelu Stable Diffusion kodują informacje o głębokości sceny i segmentacji semantycznej – możliwe jest ich odczytanie za pomocą prostej sondy liniowej. Niniejszy projekt rozszerza to podejście na analizę reprezentacji oświetlenia.

Surkov et al. [5] przeanalizowali model SDXL Turbo i wykazali, że poszczególne warstwy transformera wyspecjalizowały się w różnych aspektach obrazu: kompozycji, detalach oraz kolorze i oświetleniu. Potwierdza to, że informacja o oświetleniu jest obecna i lokalizowalna w konkretnych warstwach modelu.

4.2. Świadomość 3D w modelach wizyjnych

Banani et al. [4] zbadali, czy modele wizyjne (w tym Stable Diffusion) posiadają wewnętrzną reprezentację przestrzeni 3D. Używając sond liniowych do predykcji głębokości i normalnych powierzchni, wykazali, że modele dyfuzyjne kodują bogatsze informacje 3D niż modele dyskryminatywne. Metodologia i podział datasetu według kategorii obiektów zastosowane w tej pracy stanowią wzorzec dla niniejszego projektu.

4.3. Liniowe sondy jako narzędzie analizy

Alain i Bengio [6] zaproponowali metodę sond liniowych do badania wewnętrznych reprezentacji sieci neuronowych. Kluczowe założenie jest proste: jeśli informacja jest liniowo dekodowalna z aktywacji danej warstwy, to jest w niej jawnie zakodowana. Metoda ta pozwala odróżnić rzeczywiste kodowanie zmiennych fizycznych od przypadkowej korelacji z wzorcami teksturowymi.

4.4. Architektura modelu SANA

Xie et al. [2] przedstawiają SANA jako efektywny model dyfuzyjny oparty na liniowym Diffusion Transformerze (Linear DiT). Model wykorzystuje głęboki autoenkoder DC-AE kompresujący obraz $32\times$, co znacznie przyspiesza inferencję. SANA-0.6B jest ponad $100\times$ szybszy od FLUX-12B przy porównywalnej jakości na benchmarku DPG-Bench. W projekcie wykorzystana zostaje wersja SANA-1.6B oferująca wyższą jakość generowania.

5. Plan realizacji projektu

5.1. Etap 1 – Przygotowanie zbioru danych

- Modyfikacja skryptu `render_images.py`: sterowanie kątem lampy, wyłączenie świateł pomocniczych, wymuszenie dużego rozmiaru obiektów.
- Skrypt PowerShell do automatycznego generowania obrazów z losowo dobieranymi kątami padania światła.
- Weryfikacja poprawności plików JSON (pola `light_azimuth_deg`, `light_elevation_deg`) oraz wizualna kontrola jakości renderów.
- Wygenerowanie pełnego zbioru 800 obrazów.

5.2. Etap 2 – Konfiguracja środowiska obliczeniowego

- Przesłanie zbioru danych na superkomputer Athena.
- Konfiguracja środowiska: Python 3.10+, PyTorch 2.x, `diffusers`, `scikit-learn`.
- Przygotowanie skryptów SLURM do kolejgowania zadań GPU.
- Weryfikacja dostępu do modelu SANA-1.6B za pośrednictwem biblioteki `diffusers`.

5.3. Etap 3 – Ekstrakcja aktywacji modelu

- Załadowanie modelu SANA-1.6B z biblioteki `diffusers`.
- Implementacja mechanizmów *forward hooks* na wybranych blokach liniowego transformera (wczesnych, środkowych, późnych).
- Dla każdego obrazu: kodowanie do przestrzeni latentnej, dodanie szumu, forward pass z `torch.no_grad()`.

5.4. ETAP 4 – TRENING LINIOWEJ SONDY

- Ekstrakcja aktywacji dla trzech kroków czasowych.
- Spłaszczenie tensorów i zapis do plików `.npy`.

5.4. Etap 4 – Trening liniowej sondy

- Przygotowanie wektorów cech z opcjonalną redukcją wymiarowości metodą PCA.
- Zastosowanie reprezentacji cyklicznej azymutu: predykcja pary $(\sin \phi, \cos \phi)$.
- Trening regresji Ridge (scikit-learn) osobno dla każdej warstwy i każdego kroku t .
- Ewaluacja na zbiorze walidacyjnym (walce, kule) oraz testowym (sześciany).
- Porównanie z punktami odniesienia: baseline losowy, baseline średniej i sonda na pikselach.

5.5. Etap 5 – Analiza wyników

- Obliczenie MAE kąтового dla azymutu i wysokości dla poszczególnych warstw i kroków t .
- Heatmapa MAE (oś X: krok czasowy, oś Y: warstwa modelu).
- Wykres rozrzutu: predykcja vs. ground-truth dla najlepszej konfiguracji.
- Analiza generalizacji: porównanie MAE na znanych kształtach oraz sześcianach.
- Interpretacja wyników w kontekście postawionej hipotezy badawczej.

5.6. Etap 6 – Raport końcowy

- Uzupełnienie dokumentacji o wyniki eksperymentów i wnioski.
- Przygotowanie wizualizacji: heatmapy, wykresy rozrzutu, przykładowe obrazy z nałożonymi predykcjami.
- Sformułowanie wniosków dotyczących postawionej hipotezy.

6. Plan testów i walidacji

6.1. Metryki oceny

Dla każdej predykcji porównuje się przewidziany kąt z wartością zapisaną w pliku JSON. Podstawową miarą błędu jest MAE (*Mean Absolute Error*) wyrażone w stopniach.

Dla azymutu (kąt poziomy, cykliczny $0^\circ - 360^\circ$):

$$\text{MAE}_\phi = \frac{1}{N} \sum_{i=1}^N \min(|\hat{\phi}_i - \phi_i|, 360^\circ - |\hat{\phi}_i - \phi_i|) \quad (6.1)$$

Dla wysokości (kąt pionowy):

$$\text{MAE}_\theta = \frac{1}{N} \sum_{i=1}^N |\hat{\theta}_i - \theta_i| \quad (6.2)$$

6.2. Punkty odniesienia

Przed oceną wyników sondy ustala się trzy poziomy odniesienia odzwierciedlające jakość predykcji bez wykorzystania modelu:

Tabela 6.1: Punkty odniesienia dla azymutu

Baseline	Opis
Losowy	Przewidywanie losowego kąta
Zawsze średnia	Przewidywanie średniej kąta ze zbioru treningowego
Sonda na aktywacjach	Cel projektu - regresja na aktywacjach SANA

Wynik sondy na aktywacjach przekraczający punkty odniesienia stanowi dowód na to, że model koduje informację o oświetleniu.

6.3. Testy

Test podstawowy. Ocena MAE na zbiorze walidacyjnym (sześciany i kule, niewidziane kąty). Weryfikuje, czy sonda skutecznie dekoduje kierunek oświetlenia.

Test generalizacji. Ocena MAE na zbiorze testowym (sześciany niewidziane podczas treningu). Jest to kluczowy test hipotezy: porównywalne wyniki z testem podstawowym świadczą o tym, że sonda nauczyła się reprezentacji fizycznej, a nie cech specyficznych dla konkretnego kształtu.

Wpływ wyboru warstwy. Porównanie MAE dla wczesnych, środkowych i późnych bloków transformera. Odpowiada na pytanie, w której warstwie modelu SANA informacja o oświetleniu jest najdostępniejsza.

Wpływ kroku czasowego. Porównanie MAE dla różnych kroków czasowych t . Odpowiada na pytanie, na którym etapie procesu *denoising* reprezentacja oświetlenia jest najsilniejsza.

7. Wykorzystane technologie

Tabela 7.1: Zestawienie technologii wykorzystanych w projekcie

Technologia	Zastosowanie
Blender 2.78c	Renderowanie zbioru danych
Python 3.10+	Główny język projektu
PyTorch 2.x	Obsługa modelu dyfuzyjnego, ekstrakcja aktywacji
Hugging Face diffusers 0.27+	Ładowanie SANA-1.6B i forward pass
scikit-learn 1.x	Regresja Ridge, PCA, metryki
NumPy / SciPy	Obliczenia numeryczne, metryki kątowe
Matplotlib / Seaborn	Wizualizacje (heatmapy, wykresy rozrzutu)
PowerShell 5.1	Automatyzacja renderowania (Windows)
Superkomputer Athena	Ekstrakcja aktywacji i trening sondy (GPU, SLURM)
SLURM	System kolejkowania zadań GPU
Git / GitHub	Kontrola wersji kodu i dokumentacji

Model SANA-1.6B wymaga około 16 GB pamięci VRAM [2], co pozwala na uruchomienie na infrastrukturze HPC superkomputera Athena.

Bibliografia

- [1] Johnson, J., Hariharan, B., van der Maaten, L., Fei-Fei, L., Zitnick, C. L., Girshick, R. (2017). *CLEVR: A Diagnostic Dataset for Compositional Language and Elementary Visual Reasoning*. CVPR 2017. arXiv:1612.06890. <https://arxiv.org/abs/1612.06890>
- [2] Xie, E., Chen, J., Chen, J., Cai, H., Tang, H., Lin, Y., Zhang, Z., Li, M., Zhu, L., Lu, Y., Han, S. (2024). *SANA: Efficient High-Resolution Image Synthesis with Linear Diffusion Transformers*. arXiv:2410.10629. <https://arxiv.org/abs/2410.10629>
- [3] Chen, Y., Viégas, F., Wattenberg, M. (2023). *Beyond Surface Statistics: Scene Representations in a Latent Diffusion Model*. NeurIPS 2023 Workshop on Diffusion Models. arXiv:2306.05720. <https://arxiv.org/abs/2306.05720>
- [4] Banani, M., Raj, A., Maninis, K., Kar, A., Li, Y., Rubinstein, M., Sun, D., Guibas, L., Johnson, J., Jampani, V. (2024). *Probing the 3D Awareness of Visual Foundation Models*. CVPR 2024. arXiv:2404.08636. <https://arxiv.org/abs/2404.08636>
- [5] Surkov, V., Wendler, C., Mari, A., Terekhov, M., Deschenaux, J., West, R., Gulcehre, C., Bau D. (2024). *One-Step is Enough: Sparse Autoencoders for Text-to-Image Diffusion Models*. arXiv:2410.22366. <https://arxiv.org/abs/2410.22366>
- [6] Alain, G., Bengio, Y. (2016). *Understanding intermediate layers using linear classifier probes*. arXiv:1610.01644. <https://arxiv.org/abs/1610.01644>